DevPulse API

Coding Standards & Engineering Guidelines

Version: 1.0
Status: Approved
Compliance: Mandatory
Related Documents: - 02-software-architecture.md - 06-api-standards.md - 11-architecture-decisions.md

---

1. Purpose

تحدد هذه الوثيقة المعايير الرسمية لكتابة الكود داخل مشروع **DevPulse API**.

أي كود لا يلتزم بهذه الوثيقة يعتبر غير مطابق لمعايير المشروع.

هذه الوثيقة مرجع إلزامي أثناء:

- التطوير.
- مراجعة الكود.
- Refactoring.
- التدريب.

---

2. Engineering Principles

المشروع يلتزم بالمبادئ التالية:

- SOLID
- DRY
- KISS
- YAGNI
- Convention over Configuration
- Explicit over Implicit
- Readability First

---

3. Project Structure

```
app/

├── Http/
│   ├── Controllers/
```

```
│   ├── Requests/
│   ├── Resources/
│
├── Models/
│
├── Services/
│
├── Policies/
│
├── Exceptions/
│
├── Actions/ (Future)
│
├── Jobs/ (Future)
│
└── Support/
```

أي Folder جديد يجب أن يوثق قبل اعتماده.

---

4. Controllers

MUST

- تبقى صغيرة.
- تحتوي على أقل قدر ممكن من المنطق.
- تعتمد على Services.

MUST NOT

- تنفيذ Business Logic.
- كتابة Queries معقدة.
- Validation داخل Controller.
- File Upload Logic.
- Authorization Logic اليدوي.

---

5. Form Requests

كل Endpoint يستقبل بيانات

MUST

- يمتلك Form Request مستقل.

---

6. Services

كل Business Logic

MUST

أن يكون داخل Service.

مثال

```
PostService

AuthService

CommentService
```

الخدمة يجب أن تنفذ منطق العمل فقط، ولا تتعامل مع تفاصيل HTTP.

---

7. Models

يسمح داخل Model بـ:

- Relationships
- Scopes
- Accessors
- Mutators
- Casts

لا يسمح بـ:

- إرسال Emails
- Upload Files
- Business Workflows

---

8. API Resources

يحظر:

```
return $post;
```

المسموح:

```
return new PostResource($post);
```

---

9. Validation

Validation

MUST

- تتم داخل Form Requests.
- تستخدم Rules واضحة.
- تحتوي رسائل مناسبة عند الحاجة.

---

10. Authorization

Authorization

MUST

- تستخدم Policies.
- لا تكتب داخل Controller.
- تعتمد على Ownership عند الحاجة.

---

11. Database Access

المشروع يعتمد:

- Eloquent ORM.

يسمح باستخدام:

- Query Builder.

عند وجود سبب موثق.

لا يسمح باستخدام Raw SQL إلا عند وجود مبرر واضح ومراجعة مسبقة.

---

12. Naming Conventions

Classes

PascalCase

```
PostService

StorePostRequest
```

Methods

camelCase

```
createPost()

publishPost()
```

Variables

camelCase

Database

snake_case

Routes

plural

```
/posts
/categories
```

---

13. Dependency Injection

جميع Dependencies

MUST

تحقن عبر Constructor أو Method Injection.

يمنع إنشاء الكائنات يدويًا داخل Controllers إذا كان Laravel Container قادرًا على إدارتها.

---

14. Error Handling

جميع Exceptions

MUST

- تعالج مركزياً.
- تعيد JSON فقط.
- لا تكشف تفاصيل داخلية.

---

15. Logging

يسجل فقط:

- الأخطاء غير المتوقعة.
- الأحداث المهمة عند الحاجة.

يمنع تسجيل:

- كلمات المرور.
- Tokens.
- البيانات الحساسة.

---

16. Comments

المبدأ:

> الكود الواضح أفضل من كثرة التعليقات.

يكتب تعليق فقط إذا كان يشرح قرارًا غير بديهي، وليس ما يفعله السطر.

---

17. Testing

كل Feature

MUST

- تمتلك Feature Test.

كل Business Logic معقد

SHOULD

- يمتلك Unit Test عند الحاجة.

---

18. Code Review Checklist

قبل قبول أي Pull Request:

- هل Controller نحيف؟
- هل Validation في Form Request؟
- هل يوجد Service؟
- هل Resource مستخدم؟
- هل Policy مطبقة؟
- هل توجد اختبارات؟
- هل Naming صحيح؟
- هل الكود مقروء؟
- هل توجد أي مخالفة للمعمارية؟

---

19. Definition of Done (Code)

لا يعتبر الكود منتهيًا حتى:

- يمر جميع الاختبارات.
- يلتزم بجميع المعايير.
- يحدث التوثيق عند الحاجة.
- يراجع ويوافق عليه.

---

20. Future Standards

قد تضاف لاحقًا:

- Repository Pattern (إذا ظهرت حاجة حقيقية).
- Action Classes.
- DTOs.
- Domain Events.
- CQRS.

لن تعتمد أي تقنية جديدة إلا بعد توثيق قرارها في **Architecture Decisions**.

21. Revision History

Version	Date	Description
1.0	2026-07-21	Initial Coding Standards